

¿Qué tan modificable es el reporte de Cucumber integrado con Playwright?

Análisis de personalización, limitaciones, alternativas y validación práctica (con evidencia real).

Objetivo

Determinar qué se puede modificar: presentación, estructura y contenido (imágenes, evidencias, estilos, info).

Caso

Una empresa quiere agregar imágenes al reporte de Cucumber. Este documento confirma que sí es posible y cómo.

Resultado

Implementado y en ejecución: evidencia por paso con screenshots + look&feel + info de ejecución.

Objetivo y alcance

Objetivo

Analizar la capacidad de personalización del reporte generado por Cucumber integrado con Playwright, para determinar qué elementos pueden modificarse a nivel de presentación, estructura visual y contenido adicional (imágenes, evidencias, estilos, secciones informativas u otros detalles que aporten valor).

Alcance

Revisar el comportamiento actual del reporte, identificar las opciones para modificar apariencia y contenido, evaluar si se puede ajustar el look & feel, incorporar imágenes/evidencias, agregar información complementaria y definir limitaciones técnicas. También analizar alternativas si el reporte nativo tiene restricciones: configuración actual, plugins, reportes HTML personalizados o herramientas adicionales.



RECOMENDACIÓN — Veredicto

El reporte es ALTAMENTE modificable en contenido por paso (incluidas imágenes/evidencias) y en look & feel/información. Lo único rígido es la estructura del dashboard y los gráficos. Agregar imágenes —lo que pide la empresa— es totalmente viable y ya está implementado.

Cómo se genera y personaliza el reporte

El reporte HTML lo produce `multiple-cucumber-html-reporter` a partir del JSON de Cucumber. La personalización ocurre en dos niveles:

Nivel 1 — Contenido por paso (adjuntos)

Durante la prueba, cada paso puede "adjuntar" evidencia con `attach(dato, tipo)`. El reporter la renderiza según el tipo MIME. Aquí está la clave para agregar imágenes: adjuntamos HTML propio con la captura embebida.

Tipo de adjunto	Cómo lo muestra el reporte
<code>text/html</code>	Inyecta el HTML tal cual (tarjetas con imagen, código, estado)
<code>image/png</code>	Imagen <code></code>
<code>video/webm</code>	Video embebido
<code>text/plain</code>	Texto (rutas, logs, URL al fallar)
<code>application/json</code>	Bloque JSON

En este framework, cada acción del Page Object toma un screenshot y adjunta una "tarjeta" HTML (badge del paso, descripción, código ejecutado y la imagen en base64). Por eso cada Given/When/Then muestra su evidencia visual.

Nivel 2 — Reporte global (opciones del reporter)

Opción	Qué personaliza
<code>customStyle</code>	CSS propio para todo el reporte (colores, fuentes, banda de marca)
<code>pageFooter</code>	Pie de página propio (branding)
<code>customData</code>	Bloque de información de ejecución (proyecto, ambiente, ejecutor, fecha)
<code>metadata</code>	Chips de entorno (browser, device, plataforma)
<code>reportName / pageTitle</code>	Título y nombre del reporte
<code>useCDN</code>	Cargar librerías desde CDN (evita copiar assets)

Elementos modificables y no modificables

Elemento	¿Modificable?	Cómo
Imágenes / evidencias por paso	Sí (total)	attach de HTML con screenshot base64
Screenshots, código, badges, colores por paso	Sí (total)	HTML propio (renderCard)
Video del escenario	Sí	adjunto video/webm
Información de ejecución	Sí	customData / customMetadata
Look & feel (colores, fuentes, logo)	Sí	customStyle / overrideStyle
Pie de página / branding	Sí	pageFooter
Estructura del dashboard (columnas, layout)	Limitado	Solo forkeando plantillas (frágil)
Gráficos doughnut (tipo/colores)	No (por opción)	Fijos en plantilla (Chart.js v2)
JavaScript propio en los adjuntos	No	El reporter inyecta HTML/CSS estático

Limitaciones técnicas

- Los adjuntos deben ser HTML autocontenido: sin CSS/JS/fuentes externas (todo inline o base64).
- No se ejecuta JavaScript de los adjuntos: no hay interactividad propia por paso.
- La estructura del reporter y los gráficos vienen de sus plantillas; cambiarlas implica forkear (no mantenible).
- La copia de assets (jQuery/Bootstrap/Chart.js) puede fallar en carpetas OneDrive; mitigado copiándolos en report.ts o usando useCDN.

Alternativas disponibles

Opción	Personalización	Esfuerzo	Cuándo
Opciones del reporter actual	Media-alta	Bajo	Branding + info (hoy)
Adjuntos text/html (implementado)	Máxima (por paso)	Bajo	Evidencia visual rica
Forkear plantillas del reporter	Alta (estructura)	Alto/frágil	Solo si es imprescindible
@cucumber/html-formatter (oficial)	Media	Bajo	Reporte oficial, robusto
Allure (allure-cucumberjs)	Muy alta	Medio	Historial, tendencias, categorías
Reporte HTML propio	Total	Alto	Necesidad muy específica
PDF por escenario (implementado)	Total	Ya hecho	Evidencia portable/formal



RECOMENDACIÓN — Recomendación

Mantener multiple-cucumber-html-reporter + adjuntos text/html (ya entrega imágenes y evidencia por paso) y usar customStyle + customData + pageFooter para look&feel e información. Si a futuro se necesita reportería avanzada (historial, tendencias, categorización de fallos) se recomienda migrar a Allure. El PDF por escenario ya cubre evidencia portable.

Validación práctica — reporte de ejemplo

Todo lo anterior está implementado y en ejecución en el proyecto. A continuación, la evidencia real tomada de la última corrida.

1) Inclusión de imágenes / evidencias

Cada paso del escenario muestra una tarjeta con: número de paso, tipo de acción, descripción, el código ejecutado y una CAPTURA de pantalla. Ejemplo real (extraído del reporte):

#1 NAVIGATE

Página de Acceso al home con usuario valido cargada

page.goto('https://opensource-demo.orangehrmlive.com/web/index.php/auth/login')



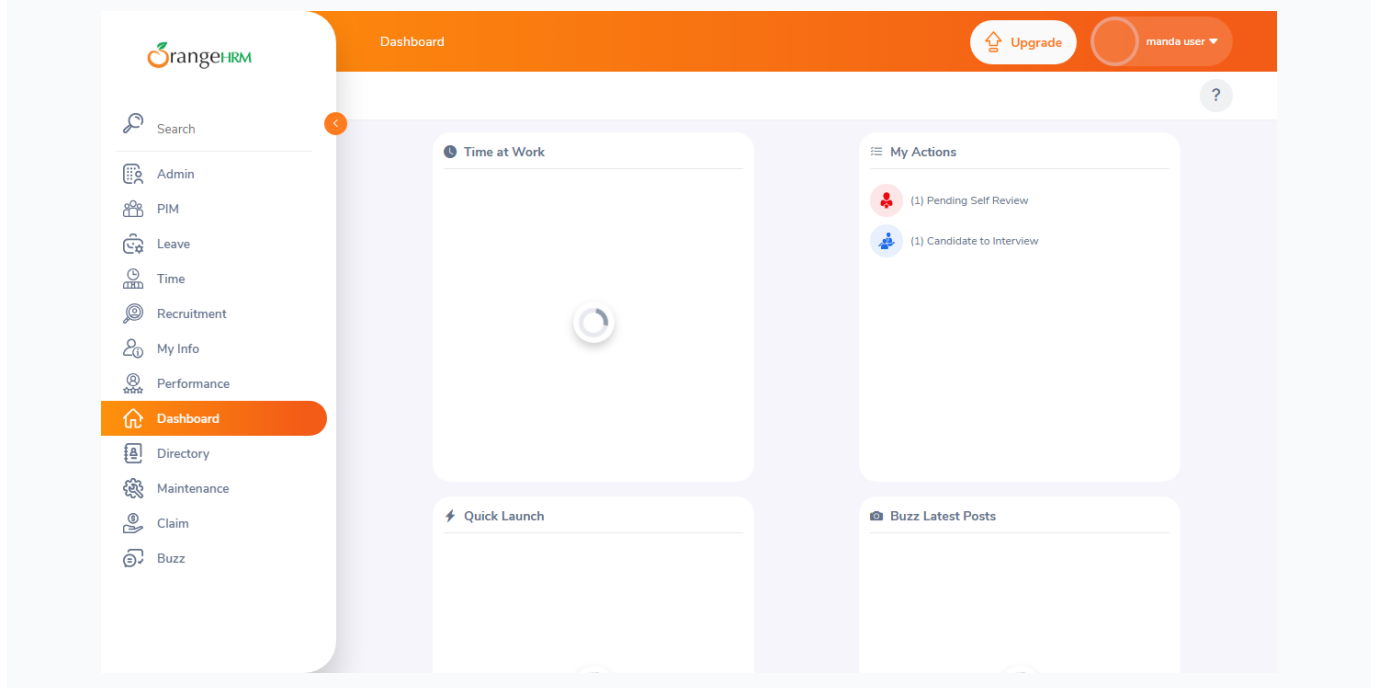
Y una verificación (ASSERT) con su captura y estado:

#5 ASSERT

verifica si el usuario llego al home page '



```
expect(locator).toBeVisible()
```



2) Información adicional de la ejecución (customData)

En la cabecera del reporte se agrega un bloque "Run info" con datos de la corrida:

Campo	Valor de ejemplo
Project	Workflow
Release	1.0.0
Environment	QA
SDET Engineer	Ezequiel Medina Adames
Executed	fecha y hora de la corrida

3) Cambios de estilo / look & feel

Se inyecta un CSS propio (customStyle) y un pie de página (pageFooter). Ejemplo aplicado:

```
body::before { content: 'QA Automation – Playwright + Cucumber'; background: #4f46e5; color:#fff; ... }
```

- customStyle -> banda de marca superior + acento de color corporativo + tipografía.
- pageFooter -> pie de página con branding del framework.
- customData -> bloque de información de ejecución (arriba).



NOTA — Reproducible

Correr: `npm run test:qa`. Abrir `reports/html/index.html` con recarga forzada (Ctrl+Shift+R). Los archivos de ejemplo son `core/reports/report-theme.css` y `report-footer.html`, cableados en `report.ts`.

Conclusión

El reporte de Cucumber + Playwright es altamente personalizable. Se pueden incorporar imágenes y evidencias por paso (vía adjuntos HTML con screenshots base64), ajustar el look & feel (customStyle/pageFooter) y agregar información de ejecución (customData). Las únicas restricciones son la estructura del dashboard y los gráficos, resolubles con Allure si el negocio lo requiere. La necesidad de la empresa — agregar imágenes al reporte— es viable y está demostrada con evidencia real en este documento.